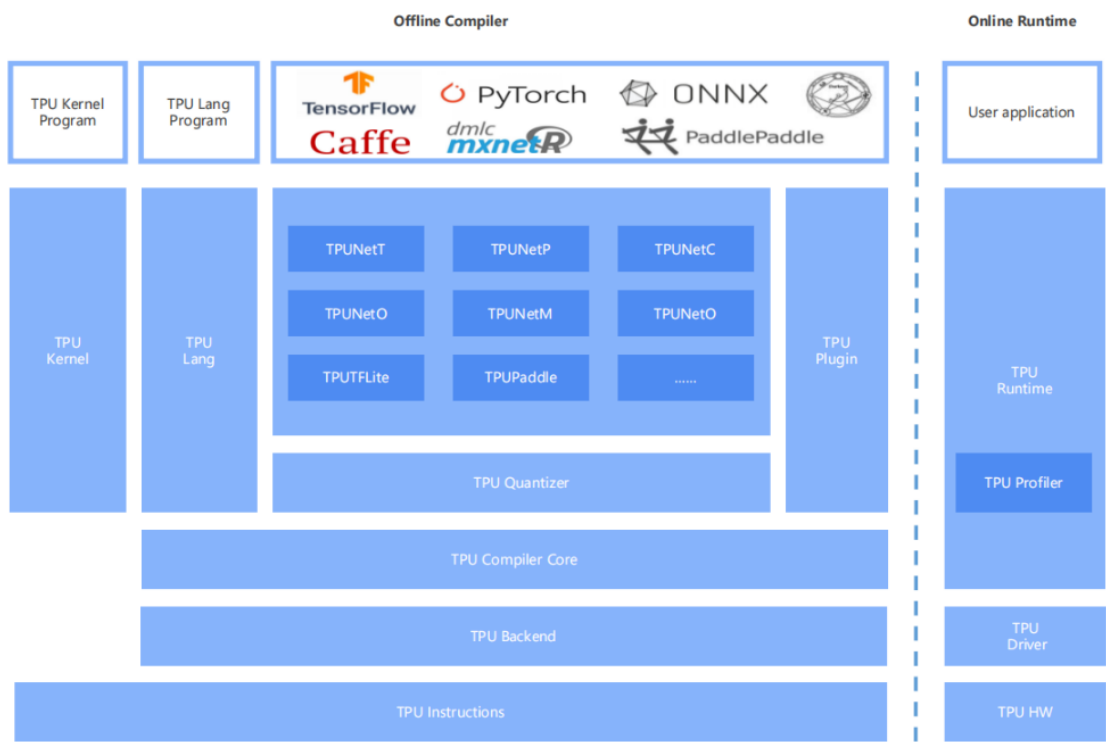
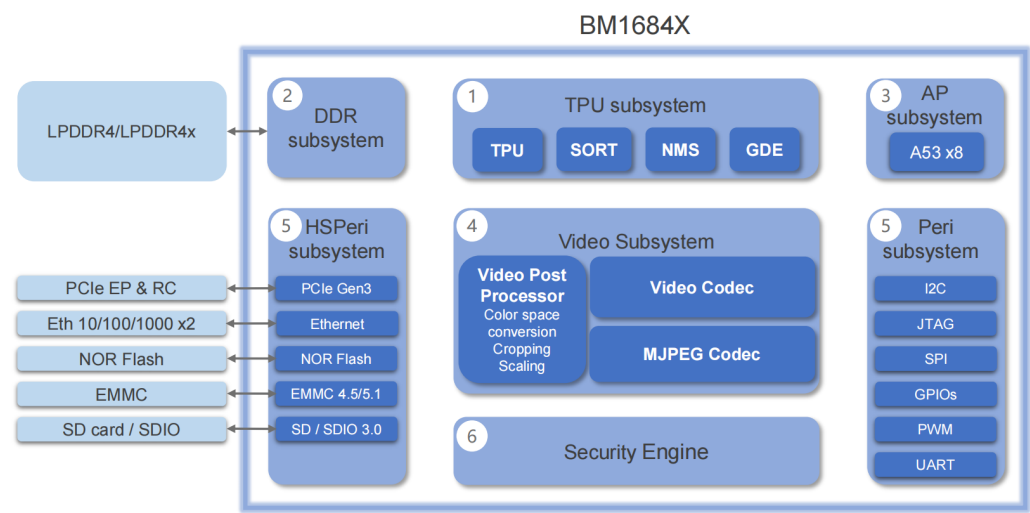


BM1684X开发

整体感知

SOPHON BM1684X 功能框图



SDK开发包

文件夹名	备注
libsophon	支持sophon设备硬件加速的图像处理、张量运算、模型推理等库
sophon-mw	支持sophon设备硬件加速的多媒体库
tpu-kernel	底层开发接口
tpu-mlir	TPU编译器工具链
tpu-nntc	TPU编译器工具链
tpu-perf	模型性能和精度验证工具包
sophon-pipeline	基于pipeline的高性能推理框架
sophon-img	SoC模式安装包
sophon-demo	针对单模型或者场景的综合例程
sophon-sail	对底层接口进行C++/Python API封装的接口库
sophon-rpc	在PCIe卡产品上将一些计算任务offload到卡上的ARM CPU 上进行

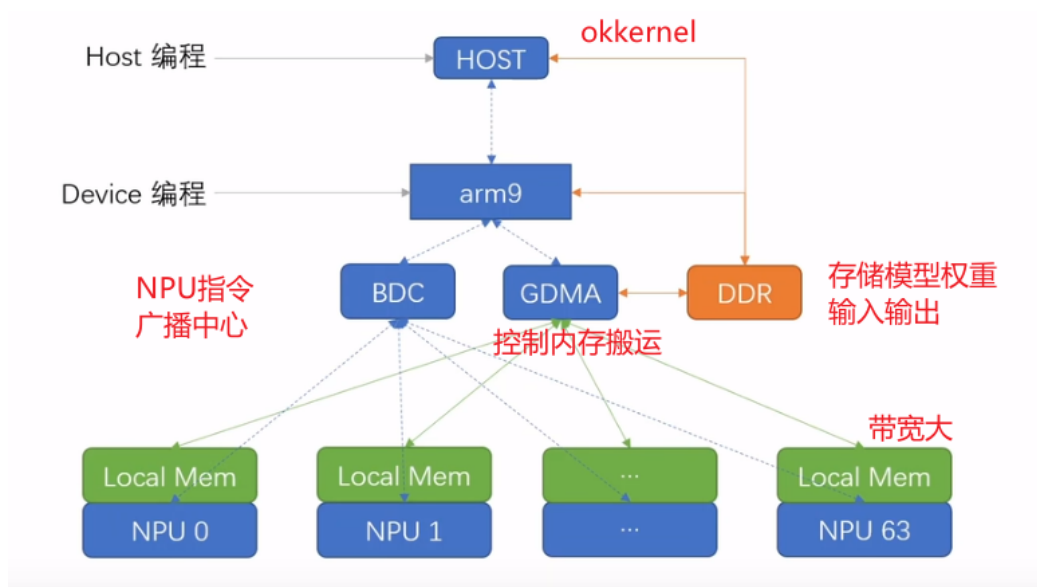
环境配置

参考sc7fp300环境搭建

https://doc.sophgo.com/sdk-docs/v23.07.01/docs_latest_release/docs/SophonSDK_doc/zh/html/index.html

开发原理

kernel原理

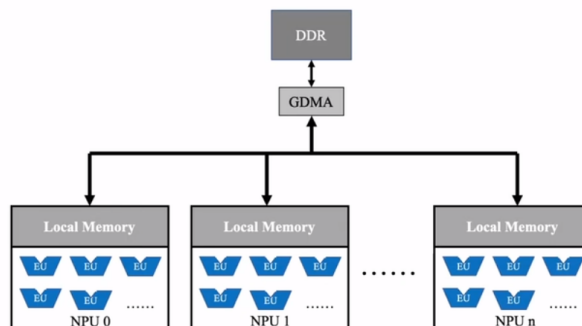


okkernel: 提供底层原子接口，封装通讯机制

全局DDR核本地内存的关系 (bm1684) 单指令 多数据,

众核架构

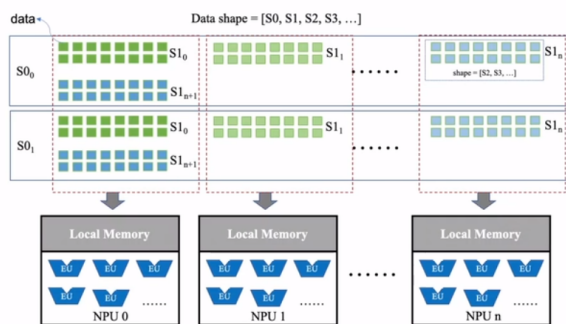
每个核称作一个神经网络处理单元(NPU)，每个NPU都有独立的本地存储(Local Memory)和多个计算单元(EU)。所有NPU以单指令多数据(SIMD)方式完成一个计算操作，只有在NPU本地存储中的数据才能被EU访问



并发模式

SOPHGO 算

- 多 TPU 并发
一共 64 个 NPU local memory 中的数据格式完全，单指令可以并发处理

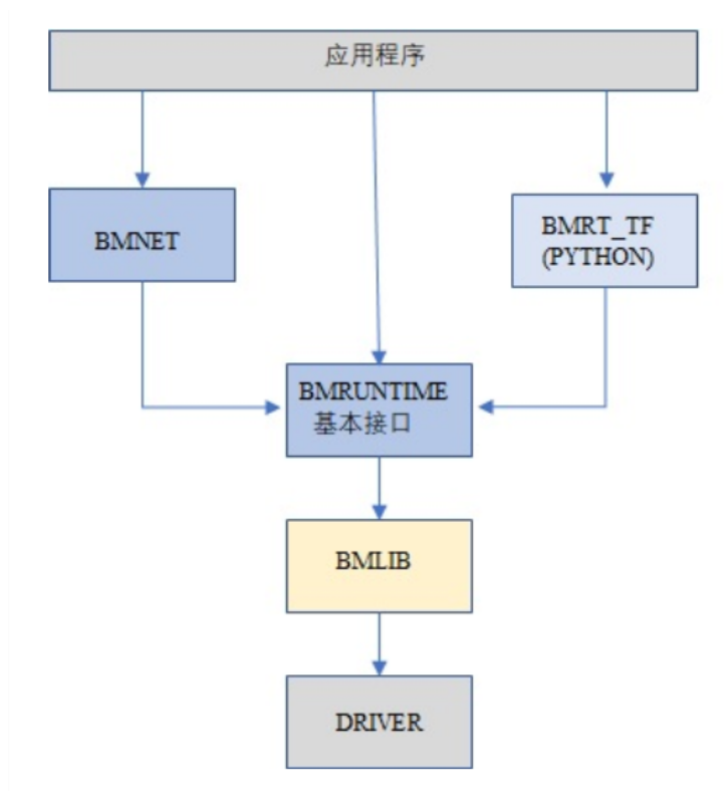


原子操作在arm9上执行

BMRuntime

总结：读取bmodel，将bmodel放到tpu上执行，提供接口移植算法。

BMRuntime用于**读取BMCompiler的编译输出(.bmodel)**，驱动其在SOPHON TPU芯片中执行。BMRuntime向用户提供了丰富的接口，便于用户移植算法，其软件架构如下：



提供两种接口 c / c++，接口默认都是同步接口，有个别是异步接口(由NPU执行功能，CPU可以继续往下执行)，会特别说明。

BMRuntime分为四个部分：

- BMLIB接口：用于设备管理，不属于BMRuntime，但需要配合使用，所以先介绍
- C接口：BMRuntime的C语言接口
- C++接口：BMRuntime的C++语言接口
- 多线程编程：介绍如何用C接口或者C++接口进行多线程编程

BMLIB

对应的头文件是bmlib_runtime.h，对应的lib库为libbmlib.so，用于**设备管理，包括设备内存的管理**

常用

1.设备

bm_dev_request

```
1  /* [out] handle
2  * [in]  devid
3  */
4  bm_status_t bm_dev_request(bm_handle_t *handle, int devid);
```

用于请求一个设备，得到设备句柄handle。其他设备接口(bm_xxxx类接口)，都需要指定这个设备句柄。

参数名	输入/输出	说明
handle	输出	保存创建的handle的指针
devid	输入	指定具体设备

其中devid表示设备号，在PCIE模式下，存在多个设备时可以用于选择对应的设备；在SoC模式下，请指定为0。

当请求成功时，返回BM_SUCCESS；否则返回其他错误码。

bm_dev_free

```
1  /* [out] handle
2  */
3  void bm_dev_free(bm_handle_t handle);
4
5  //使用参考
6  // start program
7  bm_handle_t bm_handle;
8  bm_dev_request(&bm_handle, 0);
9  // do things here
10 .....
11 // end of program
12 bm_dev_free(bm_handle);
```

用于释放一个设备。通常应用程序开始需要请求一个设备，退出前释放这个设备。

bmrt_load_bmodel

```
1  /*
2  Parameters: [in] p_bmrt      - Bmruntime that had been created.
3              [in] bmodel_path - Bmodel file directory.
4  Returns:    bool           - true: success; false: failed.
5  */
6  bool bmrt_load_bmodel(void* p_bmrt, const char *bmodel_path);
```

加载bmodel文件，加载后bmruntime中就会存在若干网络的数据，后续可以对网络进行推理。

bmrt_load_bmodel_data

```
1  /*
2  Parameters: [in] p_bmrt      - Bmruntime that had been created.
3              [in] bmodel_data - Bmodel data pointer to buffer.
4              [in] size       - Bmodel data size.
5  Returns:    bool           - true: success; false: failed.
6  */
7  bool bmrt_load_bmodel_data(void* p_bmrt, const void * bmodel_data, size_t
size);
```

加载bmodel，不同于bmrt_load_bmodel，它的bmodel数据存在内存中。

2.内存

bm_malloc_device_byte

```
1  /* [in] handle
2  * [out] pmem
3  * [in] size
4  */
5  bm_status_t bm_malloc_device_byte(bm_handle_t handle, bm_device_mem_t *pmem,
6  unsigned int size);
```

申请指定大小的device mem，size为device mem的字节大小。当申请成功时，返回BM_SUCCESS；否则返回其他错误码。

bm_free_device

```
1  /* [in] handle
2  * [out] mem
3  */
4  void bm_free_device(bm_handle_t handle, bm_device_mem_t mem);
5
6  //使用参考
7  // alloc 4096 bytes device mem
8  bm_device_mem_t mem;
9  bm_status_t status = bm_malloc_device_byte(bm_handle, &mem, 4096);
10 assert(status == BM_SUCCESS);
11 // do things here
12 .....
13 // if mem will not use any more, free it
14 bm_free_device(bm_handle, mem);
```

释放device mem。任何申请的device mem，不再使用的时候都需要释放。

bm_mem_get_device_size

```
1  // [in] mem
2  unsigned int bm_mem_get_device_size(struct bm_mem_desc mem);
```

得到device mem的大小，以字节为单位。

bmrt_tensor_t

bm_tensor_t结构体用来表示一个tensor:

```
1  /*
2  bm_tensor_t holds a multi-dimensional array of elements of a single data type
3  and tensor are in device memory */
4  typedef struct bm_tensor_s {
5      bm_data_type_t dtype;
6      bm_shape_t shape;
7      bm_device_mem_t device_mem;
8      bm_store_mode_t st_mode; /* user can set 0 as default store mode */
9  } bm_tensor_t;
```

bmrt_tensor

bmrt_tensor接口可以配置一个tensor

```
1  void bmrt_tensor(bm_tensor_t* tensor, void* p_bmrt, bm_data_type_t dtype,
    bm_shape_t shape);
```

此API用于初始化张量。它将把**deviceMem**分配给 **Tensor**

因此用户应将 `bm_free_device(p_bmrt, tensor->device_mem)` 以将其释放。

初始化后, `tensor->dtype = dtype, tensor->shape = shape, and tensor->st_mode = 0`

参数名	输入/输出	说明
tensor	输出	保存bm_tensor_t指针，不为空
p_bmruntime	输入	bmruntime的指针，不为空
dtype	输入	数据类型
shape	输入	形状

bm_memcpy_s2d

将在**系统内存**上的数据拷贝到**device mem**。系统内存由void指针指定，device mem由bm_device_mem_t类型指定。拷贝成功，返回BM_SUCCESS；否则返回其他错误码。

三种分类（根据拷贝的大小和偏移）

```

1 // 拷贝的大小是device mem的大小，从src开始拷贝
2 /* [in] handle
3    * [out] dst
4    * [in] src
5    */
6 bm_status_t bm_memcpy_s2d(bm_handle_t handle, bm_device_mem_t dst, void
    *src);

```

```

1 // size指定拷贝的字节大小，从src的offset偏移开始拷贝
2 /* [in] handle
3    * [out] dst
4    * [in] src
5    * [in] size
6    * [in] offset
7    */
8 bm_status_t bm_memcpy_s2d_partial_offset(bm_handle_t handle, bm_device_mem_t
    dst,
9                                           void *src, unsigned int size,
10                                          unsigned int offset);

```

```

1 // size指定拷贝的字节大小，从src开始拷贝
2 /* [in] handle
3    * [out] dst
4    * [in] src
5    * [in] size
6    */
7 bm_status_t bm_memcpy_s2d_partial(bm_handle_t handle, bm_device_mem_t dst,
    void *src, unsigned int size);
8

```

bm_memcpy_d2s

将device mem中的数据拷贝到系统内存；拷贝成功，返回BM_SUCCESS；否则返回其他错误码。

系统内存由void指针指定，device mem由bm_device_mem_t类型指定。

三种分类（根据拷贝的大小和偏移）

```

1 // 拷贝的大小是device mem的大小，从device mem的0偏移开始拷贝
2 /* [in] handle

```

```

3  * [out] dst
4  * [in]  src
5  */
6  bm_status_t bm_memcpy_d2s(bm_handle_t handle, void *dst, bm_device_mem_t
   src);
7
8
9  // size指定拷贝的字节大小, 从device mem的offset偏移开始拷贝
10 /* [in]  handle
11  * [out] dst
12  * [in]  src
13  * [in]  size
14  * [in]  offset
15  */
16 bm_status_t bm_memcpy_d2s_partial_offset(bm_handle_t handle, void *dst,
17                                           bm_device_mem_t src, unsigned int
   size,
18                                           unsigned int offset);
19
20 // size指定拷贝的字节大小, 从device mem的0偏移位置开始拷贝
21 /* [in]  handle
22  * [out] dst
23  * [in]  src
24  * [in]  size
25  */
26 bm_status_t bm_memcpy_d2s_partial(bm_handle_t handle, void *dst,
27                                   bm_device_mem_t src, unsigned int size);

```

bm_memcpy_d2d

```

1  /* [in]  handle
2  * [out] dst
3  * [in]  dst_offset
4  * [in]  src
5  * [in]  src_offset
6  * [in]  len
7  */
8  bm_status_t bm_memcpy_d2d(bm_handle_t handle, bm_device_mem_t dst, int
   dst_offset,
9                           bm_device_mem_t src, int src_offset, int len);

```

将数据从一个device mem拷贝到另一个device mem。

dst_offset指定目标的偏移, src_offset指定源的偏移, len指定拷贝的大小。

特别注意: len是以dword为单位, 比如要拷贝1024个字节, 则len需要指定为1024/4=256。

SOC模式下接口

参考链接

https://doc.sophgo.com/sdk-docs/v23.07.01/docs_latest_release/docs/tpu-runtime/reference/html/bmruntime/runtime.html#device-memory-mmap

Program synchronize

bm_thread_sync()

同步接口。通常npu推理是异步进行的，用户的cpu程序可以继续执行。本接口用于cpu程序中确保npu推理完成。本章介绍的接口没有特别说明，都是同步接口。只有个别异步接口，需要调用bm_thread_sync进行同步。

```
1 // [in] handle
2 bm_status_t bm_thread_sync(bm_handle_t handle);
```

bmrt_launch_tensor_ex

对指定的网络，进行npu推理

[illegible]